Entity 및 Table 설계

Entity 및 Table 설계

데이터베이스에 저장될 데이터 구조를 설계합니다.

2.1. Entity 설계 (JPA 기준)

User Entity

```
@Entity
@Table(name = "users")
@Getter
@NoArgsConstructor(access = AccessLevel.PROTECTED)
@Where(clause = "deleted_at IS NULL")
public class User {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "user_id")
    private Long id;

    @Column(nullable = false, unique = true, length = 100)
    private String email;

    @Column(nullable = false)
    private String password;

    @Column(nullable = false, unique = true, length = 50)
    private String nickname;

    @Column(length = 50)
    private String region;

    @Setter
    @Column(name = "profile_image_url")
```

```
    private String profileImageUrl;

    @Column(name = "refresh_token")
    private String refreshToken;

    private String introduction;

    private LocalDateTime createdAt;
    private LocalDateTime deletedAt;

    @OneToMany(mappedBy = "owner")
    private List<Group> ownedGroups = new ArrayList<>();

    @OneToMany(mappedBy = "user")
    private List<Membership> memberships = new ArrayList<>();
}
```

Group Entity

```
@Entity
@Table(name = "groups")
@Getter
@NoArgsConstructor(access = AccessLevel.PROTECTED)
@Where(clause = "deleted_at IS NULL")
public class Group {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "group_id")
    private Long id;

    @Column(nullable = false, length = 100)
    private String name;

    @Lob
    private String description;

    @Column(length = 50)
    private String region;
```

```
    @Column(name = "max_members")
    private int maxMembers;

    @Column(name = "image_url")
    private String imageUrl;

    @Column(length = 255)
    private String tags;

    @Formula("(SELECT COUNT(1) FROM memberships m WHERE m.group_
id = group_id)")
    private int currentMembers;

    private LocalDateTime createdAt;
    private LocalDateTime updatedAt;
    private LocalDateTime deletedAt;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "owner_id")
    private User owner;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "category_id")
    private Category category;

    @Setter
    @Column(name = "cover_image_url")
    private String coverImageUrl;
}
```

Category Entity

```
@Entity
@Table(name = "categories")
@Getter
@NoArgsConstructor(access = AccessLevel.PROTECTED)
public class Category {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    @Column(name = "category_id")
    private Long id;

    @Column(nullable = false, unique = true, length = 50)
    private String name;

    @OneToMany(mappedBy = "category")
    private List<Group> groups = new ArrayList<>();
}
```

Membership Entity

```
@Entity
@Table(name = "memberships",
    uniqueConstraints = {
        @UniqueConstraint(name = "uk_user_group", columnNames = {"user_i
d", "group_id"})
    }
)
@Getter
@NoArgsConstructor(access = AccessLevel.PROTECTED)
public class Membership {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "membership_id")
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "user_id")
    private User user;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "group_id")
    private Group group;

    @Column(nullable = false, length = 20)
    private String role;

    private LocalDateTime joinedAt;
```

```
    private LocalDateTime updatedAt;
}
```

Post Entity

```
@Entity
@Table(name = "posts")
@Getter
@NoArgsConstructor(access = AccessLevel.PROTECTED)
@DynamicInsert
@DynamicUpdate
@Where(clause = "deleted_at IS NULL")
public class Post {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "post_id")
    private Long id;

    @Column(nullable = false, length = 200)
    private String title;

    @Lob
    @Column(nullable = false)
    private String content;

    @Setter
    @Column(name = "image_url")
    private String imageUrl;

    private LocalDateTime createdAt;
    private LocalDateTime updatedAt;
    private LocalDateTime deletedAt;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "user_id")
    private User user;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "group_id")
```

```
    private Group group;

    @OneToMany(mappedBy = "post", cascade = CascadeType.ALL, orphan
Removal = true)
    private List<Comment> comments = new ArrayList<>();
}
```

Comment Entity

```
@Entity
@Table(name = "comments")
@Getter
@NoArgsConstructor(access = AccessLevel.PROTECTED)
@DynamicInsert
@DynamicUpdate
@Where(clause = "deleted_at IS NULL")
public class Comment {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "comment_id")
    private Long id;

    @Lob
    @Column(nullable = false)
    private String content;

    private LocalDateTime createdAt;
    private LocalDateTime updatedAt;
    private LocalDateTime deletedAt;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "user_id")
    private User user;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "post_id")
    private Post post;

    @ManyToOne(fetch = FetchType.LAZY)
```

```
    @JoinColumn(name = "parent_id")
    private Comment parent;

    @OneToMany(mappedBy = "parent", cascade = CascadeType.ALL)
    private List<Comment> children = new ArrayList<>();
}
```

Event Entity

```
@Entity
@Table(name = "events")
@EntityListeners(AuditingEntityListener.class)
@Getter
@Setter
@NoArgsConstructor(access = AccessLevel.PROTECTED)
public class Event {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String title;

    private String description;

    private String location;

    @Column(name = "image_url")
    private String imageUrl;

    @Column(nullable = false)
    private Integer maxAttendees;

    @Column(nullable = false)
    private LocalDateTime startAt;

    @Column(nullable = false)
    private LocalDateTime endAt;
```

```
    @CreatedDate
    @Column(updatable = false)
    private LocalDateTime createdAt;

    @LastModifiedDate
    private LocalDateTime updatedAt;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "host_id")
    private User host;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "group_id")
    private Group group;

    @OneToMany(mappedBy = "event", cascade = CascadeType.ALL, orpha
nRemoval = true)
    private List<EventAttendee> attendees = new ArrayList<>();
}
```

EventAttendee Entity

```
@Entity
@Table(
    name = "event_attendees",
    uniqueConstraints = {
        @UniqueConstraint(name = "uk_event_user", columnNames = {"event
_id", "user_id"})
    }
)
@EntityListeners(AuditingEntityListener.class)
@Getter
@Setter
@NoArgsConstructor(access = AccessLevel.PROTECTED)
public class EventAttendee {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
```

```
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "event_id")
    private Event event;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "user_id")
    private User user;

    @CreatedDate
    @Column(updatable = false)
    private LocalDateTime createdAt;

    private String role;
    private String status;
}
```

GroupLike Entity

```
@Entity
@Table(name = "group_likes",
    uniqueConstraints = {
        @UniqueConstraint(name = "uk_user_group_like", columnNames = {"u
ser_id", "group_id"})
    }
)
@Getter
@NoArgsConstructor(access = AccessLevel.PROTECTED)
public class GroupLike {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "group_like_id")
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "user_id")
    private User user;

    @ManyToOne(fetch = FetchType.LAZY)
```

```
    @JoinColumn(name = "group_id")
    private Group group;
}
```

2.2. Table 설계 (SQL DDL)

테이블명: users

설명: 사용자 정보 관리

```
CREATE TABLE users (
    user_id BIGINT AUTO_INCREMENT PRIMARY KEY,
    email VARCHAR(100) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    nickname VARCHAR(50) NOT NULL UNIQUE,
    region VARCHAR(50),
    profile_image_url VARCHAR(255),
    introduction VARCHAR(255),
    refresh_token VARCHAR(255),
    created_at DATETIME(6),
    deleted_at DATETIME(6)
);
```

테이블명: groups

설명: 사용자가 생성한 모임의 기본 정보 저장

```
CREATE TABLE `groups` (
    group_id BIGINT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    description TEXT,
    region VARCHAR(50),
    max_members INT NOT NULL,
    image_url VARCHAR(255),
    tags VARCHAR(255),
    cover_image_url VARCHAR(255),
    created_at DATETIME(6),
    updated_at DATETIME(6),
```

```
    deleted_at DATETIME(6),
    owner_id BIGINT,
    category_id BIGINT,
    FOREIGN KEY (owner_id) REFERENCES users(user_id),
    FOREIGN KEY (category_id) REFERENCES categories(category_id)
);
```

테이블명: categories

설명: 그룹이 속한 분류 정보 관리

```
CREATE TABLE categories (
    category_id BIGINT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(50) NOT NULL UNIQUE
);
```

테이블명: memberships

설명: 사용자와 그룹 간의 소속 관계 관리

```
CREATE TABLE memberships (
    membership_id BIGINT AUTO_INCREMENT PRIMARY KEY,
    user_id BIGINT NOT NULL,
    group_id BIGINT NOT NULL,
    role VARCHAR(20) NOT NULL,
    joined_at DATETIME(6),
    updated_at DATETIME(6),
    FOREIGN KEY (user_id) REFERENCES users(user_id),
    FOREIGN KEY (group_id) REFERENCES `groups`(group_id),
    UNIQUE KEY uk_user_group (user_id, group_id)
);
```

테이블명: posts

설명: 그룹 내에서 작성된 게시글 정보 저장

```
CREATE TABLE posts (
    post_id BIGINT AUTO_INCREMENT PRIMARY KEY,
    title VARCHAR(200) NOT NULL,
```

```
    content TEXT NOT NULL,
    image_url VARCHAR(255),
    created_at DATETIME(6),
    updated_at DATETIME(6),
    deleted_at DATETIME(6),
    user_id BIGINT,
    group_id BIGINT NOT NULL,
    FOREIGN KEY (user_id) REFERENCES users(user_id),
    FOREIGN KEY (group_id) REFERENCES `groups`(group_id)
);
```

테이블명: comments

설명: 게시글에 대한 댓글 정보 저장

```
CREATE TABLE comments (
    comment_id BIGINT AUTO_INCREMENT PRIMARY KEY,
    content TEXT NOT NULL,
    created_at DATETIME(6),
    updated_at DATETIME(6),
    deleted_at DATETIME(6),
    user_id BIGINT,
    post_id BIGINT NOT NULL,
    parent_id BIGINT,
    FOREIGN KEY (user_id) REFERENCES users(user_id),
    FOREIGN KEY (post_id) REFERENCES posts(post_id),
    FOREIGN KEY (parent_id) REFERENCES comments(comment_id)
);
```

테이블명: events

설명: 그룹에서 생성된 일정 정보 저장

```
CREATE TABLE events (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    title VARCHAR(255) NOT NULL,
    description VARCHAR(255),
    location VARCHAR(255),
    image_url VARCHAR(255),
```

```
    max_attendees INT NOT NULL,
    start_at DATETIME(6) NOT NULL,
    end_at DATETIME(6) NOT NULL,
    created_at DATETIME(6),
    updated_at DATETIME(6),
    host_id BIGINT,
    group_id BIGINT,
    FOREIGN KEY (host_id) REFERENCES users(user_id),
    FOREIGN KEY (group_id) REFERENCES `groups`(group_id)
);
```

테이블명: event_attendees

설명: 일정에 참석하는 사용자와 참석 상태 관리

```
CREATE TABLE event_attendees (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    event_id BIGINT,
    user_id BIGINT,
    created_at DATETIME(6),
    role VARCHAR(255),
    status VARCHAR(255),
    UNIQUE KEY uk_event_user (event_id, user_id),
    FOREIGN KEY (event_id) REFERENCES events(id),
    FOREIGN KEY (user_id) REFERENCES users(user_id)
);
```

테이블명: group_likes

설명: 어떤 사용자가 어떤 그룹을 찜했는지에 대한 정보를 기록하고 관리

```
CREATE TABLE group_likes (
    group_like_id BIGINT AUTO_INCREMENT PRIMARY KEY,
    user_id BIGINT NOT NULL,
    group_id BIGINT NOT NULL,
    FOREIGN KEY (user_id) REFERENCES users(user_id),
    FOREIGN KEY (group_id) REFERENCES `groups`(group_id),
```

```
    UNIQUE KEY uk_user_group_like (user_id, group_id)
);
```